



EC8206 – Functional Programming Project Report

A Type-Safe Arithmetic Expression Interpreter in Haskell

Department of Electrical and Information Engineering
Faculty of Engineering
University of Ruhuna

EG/2021/4648 – Lorensuhewa S J

1. Project Overview

The interpreter is organized into `src/Expr.hs` for the expression language and evaluator, `app/Main.hs` for demonstration and testing, and `eval/eval_imperative.py` for the imperative comparison required in Part D.

Part A: `Expr` defines `Lit Double`, `Var String`, `Add`, `Sub`, `Mul`, `Div` and the extension constructor `Let String Expr Expr`. `Let` represents a local binding of the form `let x = e1 in e2`. `Env` is represented as `[(String, Double)]`.

Part B: `eval :: Env -> Expr -> Either String Double` evaluates the expression recursively using pattern matching. Successful computations return `Right` values, while division by zero and undefined variables return `Left` error messages instead of terminating the program.

Part C: `simplify` performs recursive algebraic rewriting. The implementation includes identities `x+0`, `0+x`, `x-0`, `x*1`, `1*x` and `x/1`. `evalBatch` evaluates a list with `map` using the partially applied function `(eval env)`, and `countResults` uses `foldr` to count successful and failed evaluations.

Requirements: GHC and Python 3 are sufficient. The project uses no external Haskell packages.

```
ghc -isrc -Wall -outputdir build -o interpreter app/Main.hs
.\interpreter.exe
ghci -isrc app/Main.hs
python eval/eval_imperative.py
```

2. Sample Evaluations

Environment used in the demonstrations: `[("x",8), ("y",4)]`. The following table records the actual GHCi results obtained during testing.

No.	Expression	Expected	Actual	Result
1	<code>6 + (2 * 5)</code>	Right 16.0	Right 16.0	PASS
2	<code>x + (y * 2)</code>	Right 16.0	Right 16.0	PASS
3	<code>let z = 10 in (z - 3)</code>	Right 7.0	Right 7.0	PASS
4	<code>10 / 0</code>	Left "Division by zero"	Left "Division by zero"	PASS
5	<code>unknown + 1</code>	Left "Undefined variable: unknown"	Left "Undefined variable: unknown"	PASS

Simplification and batch results

The simplification demonstration used the expression `(x + 0) * 1`. It was transformed to `Var "x"` and then evaluated to `Right 8.0`. The batch evaluation over the five sample expressions returned `[16.0,16.0,7.0]`, while `countResults` returned `(3,2)`, meaning three evaluations succeeded and two failed.

3. Part D - Comparative Analysis

Imperative version (`eval/eval_imperative.py`). The Python implementation uses `Expr` objects, a mutable dictionary for the environment, an `if/elif` chain to select the operation, and exceptions for error reporting. The Haskell version instead uses an algebraic data type and returns errors explicitly through `Either`.

```

def eval_expr(env, expr):
    if expr.kind == "lit":
        return expr.value
    elif expr.kind == "var":
        if expr.value in env:
            return env[expr.value]
        raise ValueError("Undefined variable: " + expr.value)
    elif expr.kind == "add":
        return eval_expr(env, expr.left) + eval_expr(env, expr.right)
    elif expr.kind == "sub":
        return eval_expr(env, expr.left) - eval_expr(env, expr.right)
    elif expr.kind == "mul":
        return eval_expr(env, expr.left) * eval_expr(env, expr.right)
    elif expr.kind == "div":
        right = eval_expr(env, expr.right)
        if right == 0:
            raise ZeroDivisionError("Division by zero")
        return eval_expr(env, expr.left) / right
    else:
        raise ValueError("Unknown expression kind: " + expr.kind)

```

Functional version (core of eval in src/Expr.hs):

```

eval :: Env -> Expr -> Either String Double
eval _ (Lit n) = Right n
eval env (Var name) =
    case lookup name env of
        Just value -> Right value
        Nothing -> Left ("Undefined variable: " ++ name)
eval env (Div e1 e2) = ...
eval env (Let name valueExpr bodyExpr) =
    case eval env valueExpr of
        Left err -> Left err
        Right value -> eval ((name, value) : env) bodyExpr

```

Error representation and propagation

In Haskell, the possibility of failure appears in the return type: eval returns `Either String Double`. A caller therefore receives either a valid numeric result or an error value and must account for that distinction. Recursive evaluation propagates a `Left` value upward without throwing an exception. In the Python version, errors are exceptions such as `ValueError` and `ZeroDivisionError`; they are outside the function's return type and are discovered when the failing input is evaluated.

Mutable state

The Python implementation uses a mutable dictionary for variable bindings and ordinary reassignment is available to the evaluator. In the Haskell version, no environment is modified. A `Let` expression creates a new environment by adding a pair to the front of the existing list and passes that new value only to the body. This preserves the original environment and naturally supports shadowing.

Bugs the type checker rules out before the program runs

The Haskell Expr type fixes the available expression constructors and their child types. A numeric literal must contain a Double, and binary arithmetic constructors require two Expr values. An operator cannot be misspelled as a string because operators are represented by constructors. Runtime-dependent cases such as an unbound variable or a zero denominator are still possible, so eval reports them using Left. With warnings enabled, GHC can also identify non-exhaustive pattern matches when new constructors are introduced.

Limitations

Python is familiar and concise for many programmers, while Haskell requires understanding algebraic data types, Either and recursive pattern matching. Neither language can determine from types alone whether a run-time environment contains a requested variable or whether a denominator evaluates to zero.

4. Part E - Reflection

Pure functions

The main functions in the project are pure: eval depends only on its environment and expression, and simplify depends only on the expression being transformed. They do not read or modify global state and do not perform input/output. Consequently, the same inputs produce the same outputs, making individual examples easy to test and reason about.

The simplify function also illustrates equational reasoning. For example, simplifying Add (Var "x") (Lit 0) produces Var "x". The transformation is expressed as a pattern-based rule rather than an instruction that mutates an existing expression tree. This makes the operation predictable and keeps the original Expr value unchanged.

Immutability

Neither Expr values nor environments are modified in place. simplify constructs a new expression tree and leaves the original tree available to the caller. Likewise, Let does not overwrite an existing binding; it creates a new list with (name, value) at the front and evaluates the body with that environment. Since lookup uses the first matching name, an inner binding shadows an outer binding only within the evaluation of that body.

Why the shape of Expr helps prevent invalid expressions

Expr is an algebraic data type, so each constructor fixes the structure and types of its components. There is no Add constructor with a missing operand and no Lit containing a string because Lit requires a Double. Operators are also explicit constructors rather than arbitrary text. This moves many structural mistakes from run time into compile time. Runtime conditions such as an undefined variable and division by zero remain value-dependent and are therefore handled by eval.

Higher-order functions and currying

The project uses the expression (eval env) as a partially applied function. The environment is fixed first, leaving a function that accepts an Expr. map then applies that function to each expression in a list. The countResults function uses foldr to accumulate the number of successful and failed evaluations. These examples connect the implementation directly to the higher-order-function and currying concepts covered in the module.

5. References

[1] G. Hutton, Programming in Haskell, 2nd ed., Cambridge University Press, 2016.

[2] Haskell.org, Haskell documentation and language resources, <https://www.haskell.org/>